

Resolución de problemas y algoritmos

Dra. Jessica Andrea Carballido

jessicarballido@gmail.com



Dpto. de Ciencias e Ingeniería de la Computación

UNIVERSIDAD NACIONAL DEL SUR

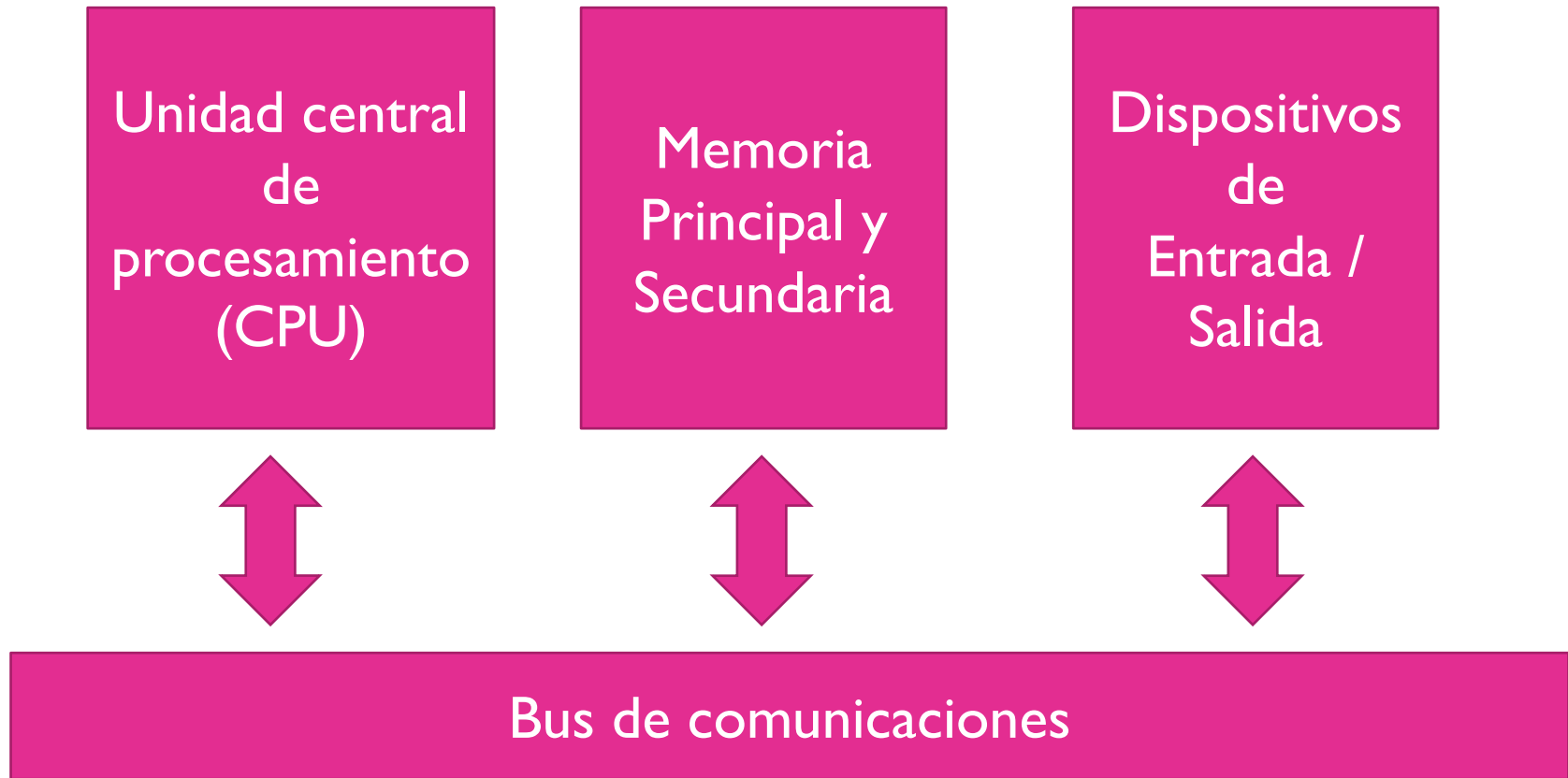
Una **computadora** es una colección de dispositivos físicos y lógicos, integrados de modo que puedan ser utilizados con propósitos muy diversos.

El **hardware** de una computadora es el conjunto de **componentes físicos**.

El **software** de una computadora es el conjunto de **componentes lógicos**.

Aunque la computadora es un dispositivos relativamente reciente, muchos de los conceptos fundamentales son muy anteriores a su invención.





VER: <https://youtu.be/-1clqsilLQ0>



Arquitectura de Von Neuman

- **Memoria:** Se almacenan programas y datos
- **CPU: Es el cerebro!** Capaz de interpretar los programas almacenados en la memoria y llevar a cabo las computaciones indicadas
- **Periféricos de entrada y salida:** Permiten al sistema comunicarse con usuarios y otros sistemas.
- **Bus de Comunicación:** Controla la transmisión de los datos desde y hacia la memoria



Es el conjunto de los **programas** y **datos** almacenados en la memoria de una computadora.

El desarrollo de software es un proceso que abarca varias etapas y demanda cubrir diferentes roles.

- **Especificación**
- **Análisis y Diseño**
- **Implementación**
- **Verificación**
- Mantenimiento

La documentación es una actividad transversal durante todo el proceso.



Una clasificación para el software es:

Sistema: Sistemas operativos, controladores, herramientas de diagnóstico, corrección y optimización, servidores, etc.

Soporte para desarrollo: Editores, compiladores, intérpretes, depuradores, IDEs.

Aplicaciones: Automatización y control industrial, Gestión, Juegos, Didácticos y Educativos, Telecomunicaciones, Diseño Asistido, Diagnóstico Asistido, etc.



Es una **secuencia de instrucciones** que pueden ser interpretadas y ejecutadas por un autómatas.

Si el autómatas es una computadora, la secuencia de instrucciones tiene que estar escrita en un **lenguaje de programación**.

Un **lenguaje de programación** es una notación formal con una **sintaxis** estricta y una **semántica** precisa.

Las reglas sintácticas establecen la **forma** de los programas y las instrucciones que lo conforman.

Las reglas semánticas describen el **significado** de las instrucciones y programas sintácticamente válidos.

Existen distintos lenguajes de programación, y distintos criterios para compararlos y seleccionar el más adecuado para cada problema particular.



El **procesador central** solo es capaz de interpretar y ejecutar **lenguaje máquina**.

El conjunto de instrucciones del lenguaje máquina es reducido y muy simple.

Los programas escritos en Pascal, o en cualquier otro lenguaje de alto nivel, deben ser **traducidos** a lenguaje máquina antes de ejecutarse.

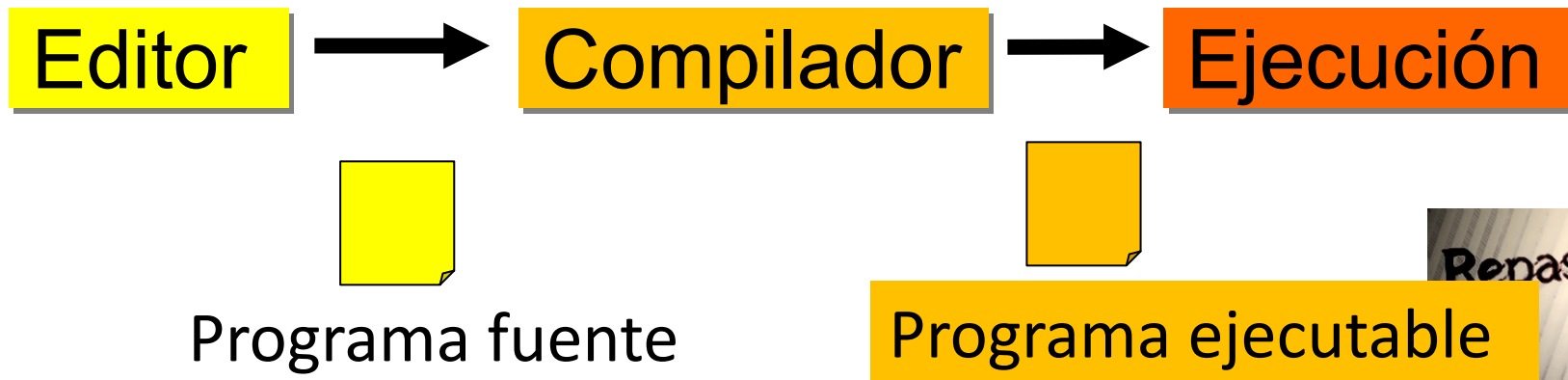
El **compilador** es un programa responsable de traducir el código escrito en un lenguaje de alto nivel a lenguaje máquina.



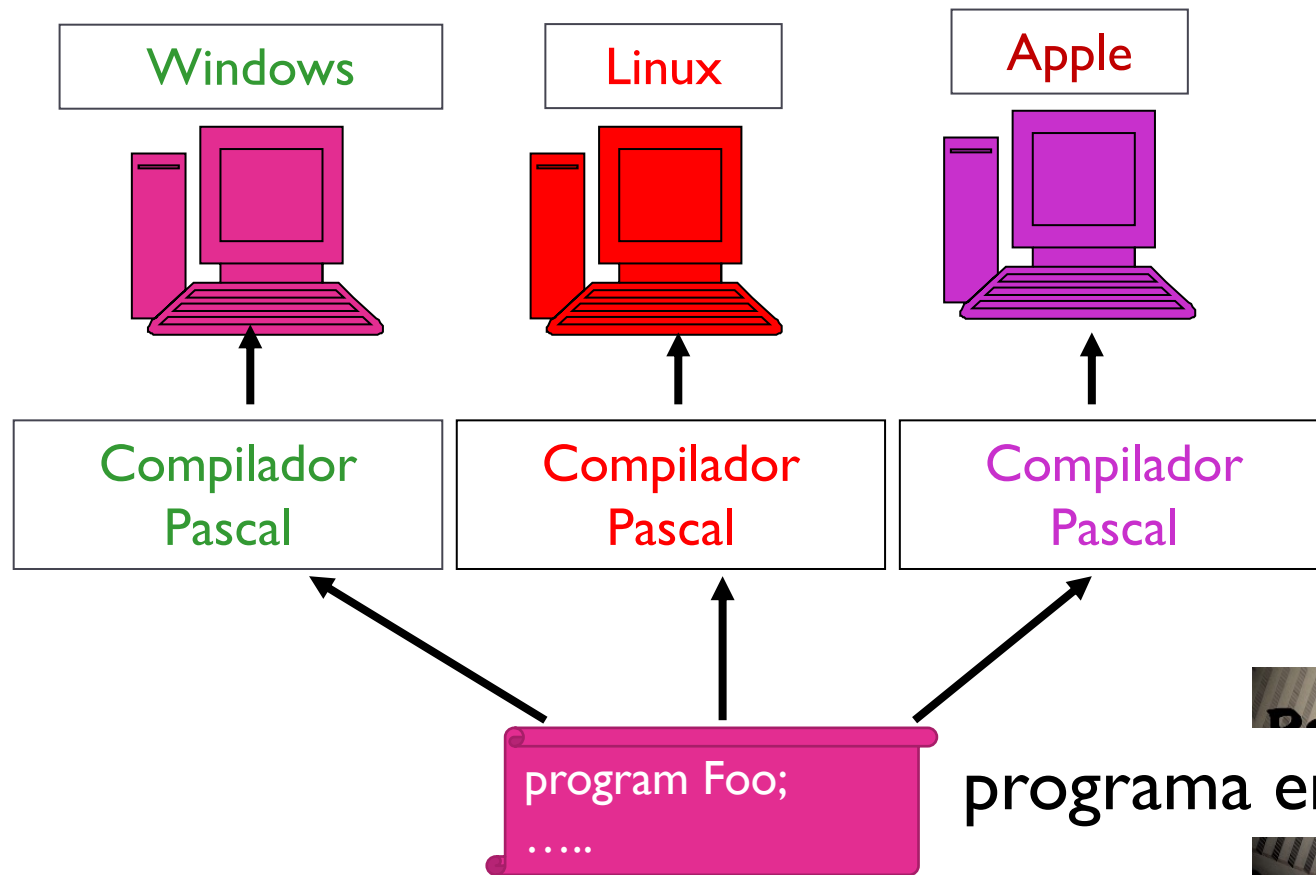
El **compilador** es también responsable de controlar la sintaxis del programa y luego traducirlo a lenguaje máquina.

El programa que escribimos se llama **programa fuente** o **código fuente**.

Una vez que se compila, obtenemos un **programa ejecutable**.



Cada **lenguaje de programación** va a tener distintos compiladores, según el sistema operativo que vaya a ejecutar los programas escritos en ese lenguaje.



programa en Pascal

Uno de los objetivos de las carreras del DCIC es formar profesionales capacitados para participar en proyectos de desarrollo de software que permitan resolver problemas complejos, muchas veces para dominios de aplicación críticos.

El principal **objetivo de RPA** es que los alumnos desarrollen competencias para escribir programas en **lenguaje** Pascal para resolver problemas simples, aplicando algunas **estrategias y metodologías** y considerando algunos **criterios de calidad**.



Pascal es un lenguaje de programación de alto nivel, adecuado para escribir programas que resuelvan problemas simples.

Aprender un lenguaje implica conocer sus reglas sintácticas y semánticas.

ANTES de escribir un programa es muy importante INTERPRETAR adecuadamente el enunciado y, si la complejidad del problema lo requiere, DISEÑAR un algoritmo.

DESPUÉS de escribir un programa es muy importante VERIFICAR que funciona correctamente para algunos casos de prueba bien elegidos.



- **Abstracción:** identificar los aspectos relevantes del problema y los que no lo son.
- **Particularización:** dado un problema general, identificar y hallar la solución de un caso específico.
- **Generalización:** dada la solución de un caso particular, hallar el patrón para la resolución de un problema más general.
- **Analogía:** encontrar una relación o semejanza entre un problema nuevo y otro ya resuelto. Dos problemas que en principio pueden parecer muy diferentes pueden tener soluciones con estructuras análogas. Dos problemas en apariencia similares pueden resolverse de manera muy diferente.



- **Dividir y Conquistar:** dividir el problema en sub-problemas más simples.
- **Refinamiento paso a paso:** diseñar algoritmos en una notación informal e ir obteniendo una secuencia de versiones hasta llegar a la implementación en un lenguaje de programación.

Ambas estrategias están orientadas a reducir la complejidad del problema para favorecer su resolución.

Se utilizan en la etapa de **diseño**.



- **Programación modular:** Utilizamos procedimientos y funciones para implementar primitivas (en Pascal) y lograr así soluciones **moduladas**, manteniendo en el código la estructura de los sub-problemas identificados en el diseño.
- **Programación estructurada:** Seleccionar adecuadamente las estructuras de control.

Ambas metodologías apuntan a obtener programas fáciles de leer, verificar y mantener.

Se utilizan en la etapa de **implementación**.



Criterios de Calidad propuestos en RPA Repaso

- **Correctitud o efectividad:** un programa es correcto si funciona de acuerdo a la especificación. No es posible garantizar que un programa es completamente correcto.
- **Legibilidad:** un programa es legible si emplea estructuras de control adecuadas, está bien indentado, utiliza identificadores significativos, incluye comentarios útiles, prescinde de datos e instrucciones irrelevantes.
- **Eficiencia:** un programa es eficiente se ejecuta de la manera más rápida posible, y aprovecha los recursos del hardware.



¿Qué es un programa?

Un programa es un conjunto de instrucciones escritas en un lenguaje de programación que una computadora puede ejecutar para realizar tareas específicas.



Identificador:

Es una combinación de letras, dígitos y el caracter '_', empezando por letra.

Es un concepto muy importante en Pascal, ya que sirve para nombrar las constantes, las variables, los subprogramas (funciones y procedimientos) e incluso el mismo programa.



Constante:

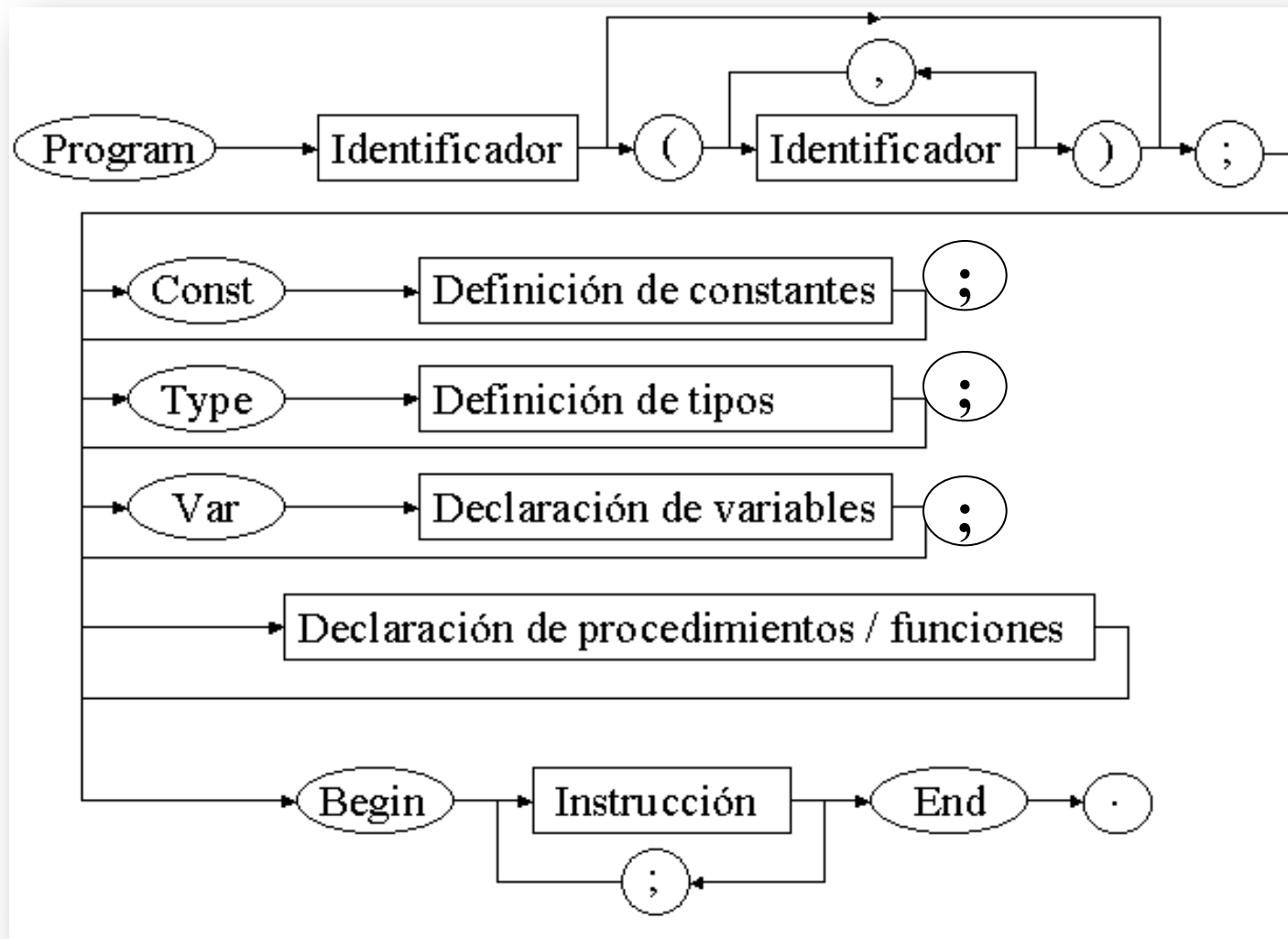
Como su nombre lo indica, permite representar un dato cuyo valor no varía a lo largo de la ejecución de todo el programa. Su tipo es inferido por Pascal.

Variable:

Es un espacio o contenedor que permite que un mismo nombre de dato almacene distintos valores a lo largo de la ejecución del programa. Tenemos que declarar su tipo explícitamente.



Diagrama de sintaxis (Conway)



Tipos de Datos

En Pascal se debe especificar a la computadora qué clase de datos va a contener una variable.

A simple vista, los humanos podemos distinguir entre una palabra como 'Hola' o un número como 47.

La computadora no puede hacer esto, así que **se le debe decir qué tipo de datos se pondrá en cada variable**. Una vez hecho esto, no puede almacenarse un valor de cualquier tipo en esa variable, sino de ese tipo o de tipos “compatibles”.



Tipos de Datos

Integer: número entero entre $-\text{MAXINT}$ y MAXINT

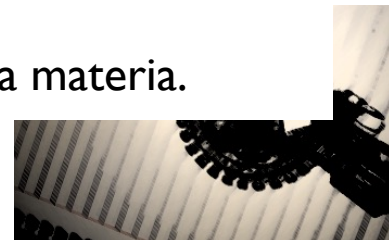
Real: número con coma decimal entre 2.9×10^{-39} y 1.7×10^{38}

Char: un caracter (un dígito o una letra o un símbolo)

Boolean: Pueden tomar dos valores, True (verdadero)
False (falso).

Estos son tipos de datos simples.

Existen muchos otros tipos de datos que quedan fuera del alcance de esta materia.



Tipos Ordinales

Un tipo ordinal representa una secuencia ordenada de valores individuales, a los que se pueden aplicar los conceptos de predecesor y sucesor (Ej. enteros, caracter, booleanos).



Ligadura entre identificadores y tipos

- Declaración **const**
const pi=3.1415926; // implícita
- Declaración **var. // explícita**
var a, b, c: integer;
- Declaración de función. // explícita, la función queda ligada a un tipo
function f(x: integer): integer;



Tipos de Datos - Compatibilidad

Pascal es un lenguaje **“fuertemente tipado”**,
los errores de tipos se detectan TODOS en compilación.

Esto implica que haya limitaciones bastante estrictas sobre cómo pueden asignarse datos de un determinado tipo a otras de tipo diferente.

En general los datos asignados a una variable deben ser del mismo tipo, pero existen algunas excepciones. Cuando se pueden asignar datos a una variable de diferente tipo, a estos tipos de datos se los denomina **“tipos de datos compatibles”**.

Tipos de Datos - Compatibilidad

Los tipos de datos compatibles entre sí son los tipos **enteros**, **reales** y **sub-rango**.

Las reglas de utilización de los tipos de datos compatibles son:

- Las variables de tipo entero pueden **asignarse** a variables de tipo real. Sin embargo, lo contrario no es posible.

~~varEntera := varReal~~ // error de compilación

varReal := varEntera

Pascal realiza una conversión implícita de tipo entero a real.

CONVERSION SEGURA
PORQUE NO SE PIERDE NADA



- Compilación
 - Operaciones con datos de tipos inadecuados
 - Errores de sintaxis
 - Variables sin declarar
- Ejecución (Provocan la terminación anormal del programa)
 - Reset de archivo inexistente
 - División por cero
 - Lectura de un dato de un tipo distinto al esperado
 - Lectura de un dato cuando se alcanzó el final de archivo
- Aplicación O LÓGICO (El programa termina normalmente, pero los resultados no son correctos.)
 - $a:=b+c$ y se quería realizar $a:=b*d$



Elementos de Pascal

Pascal es un lenguaje imperativo.

Un programa en Pascal es una secuencia de declaraciones y de **instrucciones** que se ejecutan sobre ciertos **datos**.

En RPA aprendimos un sub-conjunto de Pascal.

La mayoría de los elementos de Pascal que aprendimos van a aparecer también en Java, con una sintaxis un poco diferente.

Algunos elementos van a tener una semántica similar, en otros el significado va a ser diferente.



Instrucciones

Una **instrucción simple** en Pascal puede ser:

- Asignación
- Llamada a procedimiento
- Instrucción que depende de Condicional
- Instrucción que depende de Iteración

Una instrucción compuesta es una secuencia de instrucciones, simples o compuestas, encerradas entre las palabras reservadas begin-end.

Las instrucciones se separan por el símbolo ;



Datos

Los datos pueden ser **constantes** o **variables**.

Las constantes pueden ser literales o estar asociadas a un identificador. El tipo de la constante se **INFIERE** a partir del tipo del valor que se le asocia.

Las variables están ligadas a un identificador.

Tanto las constantes como las variables tienen asociado un tipo.

Hemos trabajado solo con algunos de los tipos provistos por Pascal.



Tipos de datos

Hemos definido variables y constantes de los **tipos simples** *integer, real, char y boolean*.

El tipo de dato *text* y el tipo “file of...” son **tipos estructurados**.

Un tipo de dato, simple o estructurado, determina un conjunto de valores y un conjunto de operaciones.

Los operadores definidos para un tipo pueden estar asociados a **símbolos** (+, /) o **identificadores** (mod, div).



Tipos de datos

Por ejemplo, para el tipo de dato *integer* el conjunto de valores es el subconjunto de los enteros entre $-\text{Maxint}$ y Maxint .

El valor de Maxint varía dependiendo del sistema operativo.

Algunos de los operadores son $+$ $-$ $*$ div mod abs sqr .

Para el tipo de dato *boolean* el conjunto de valores son las constantes predefinidas *true* y *false* y los operadores booleanos *and*, *or* y *not*.

Los operadores relacionales ($<$ $\leq$ $>$ $\geq$ $=$ $\neq$) computan un valor booleano.



Entrada y Salida

Pascal brinda cuatro procedimientos para manejar la entrada y salida, por consola o desde y hacia archivos:

- read
- readln
- write
- writeln

Todos estos procedimientos pueden invocarse con distinto número de parámetros. **Esto no se puede hacer con las primitivas definidas por el usuario.** ¿O sí? Para pensar...



Expresiones

Toda expresión está formada por **operadores y operandos** y CALCULA un valor.

Los operandos pueden ser a su vez expresiones, de modo que una expresión puede definirse **recursivamente**.

Pascal establece **reglas de precedencia** entre los operadores, las cuales indican el orden en el que van a ejecutarse las sub-expresiones.

El programador puede modificar el orden usando paréntesis.



Asignaciones

El valor computado por una expresión puede ser almacenado en una variable usando una **instrucción de asignación**.

En Pascal el operador de asignación es el símbolo compuesto **:=**

La expresión de la derecha se evalúa, se obtiene un valor y el resultado se almacena en la locación de memoria que corresponde a la variable que aparece a la izquierda.

El tipo de la variable de la izquierda de la asignación debe ser **compatible** con el tipo de la expresión que está a la derecha.

Asimismo, el tipo de los operadores de la expresión tiene que ser compatible con el tipo de los operandos.

Condicional

El condicional es una estructura de control que permite establecer el flujo de ejecución en base a una condición (expresión lógica).

Pascal brinda dos tipos de instrucción condicional:

- **if ... then ... else ...**
(pudiendo no estar la rama del else)
- **case ... of.. end**



Condicional

Cada instrucción condicional puede tener otras instrucciones condicionales anidadas, de modo que la definición de es recursiva.

La instrucción **if** se utiliza cuando las instrucciones a ejecutar dependen del valor de verdad de una **expresión booleana**.

Si la expresión computa verdadero, se ejecuta la instrucción simple o compuesta asociada al then.

Si la expresión computa falso, puede no ejecutarse ninguna instrucción (el else es opcional) o ejecutarse la instrucción simple o compuesta asociada al else



Condicional

La instrucción **case** se utiliza cuando hay dos o más instrucciones, simples o compuestas, **mutuamente excluyentes**.

La selección de qué instrucción va a ejecutarse depende del valor de una variable o expresión de algún **tipo escalar**.

Nosotros solo hemos visto tres tipos escalares: integer, char y boolean.



Iteración

Pascal brinda tres estructuras de control iterativas:

- **For ..:..To .. Do...**
- **While ... do ...**
- **Repeat ... until ...**

Cada iteración incluye una instrucción, simple o compuesta, a la que llamamos **bloque iterativo**.



Iteración for

La instrucción **for** se usa cuando se puede especificar la cantidad de veces que debe ejecutarse el bloque iterativo.

El encabezamiento del for incluye una **variable de control** y dos expresiones que determinan su valor inicial y final.

La variable de control **NO** puede modificarse en el bloque iterativo.

Los valores de las variables que aparecen en las expresiones que determinan el valor inicial y final **NO DEBEN** ser modificadas (para favorecer la legibilidad).



Iteración while do

La instrucción *while ... do* se utiliza cuando el bloque iterativo debe ejecutarse **mientras** que una condición (expresión booleana) compute el valor true.

La expresión se evalúa *antes* de cada iteración, de modo que si la primera vez computa false, el bloque iterativo no se ejecuta nunca.

La expresión booleana es la **condición de corte** de la iteración, de modo que es importante considerar la consistencia entre la inicialización de las variables, la expresión y el bloque iterativo, para garantizar que la condición de corte se satisfaga en el momento adecuado. En particular es necesario evitar los ciclos infinitos.



Iteración repeat until

La instrucción repeat until se utiliza cuando el bloque iterativo debe ejecutarse ***hasta*** que una condición (expresión booleana) compute el valor true.

La expresión se evalúa después de cada iteración, de modo que el bloque iterativo **se ejecuta al menos una vez**.

La expresión booleana es la **condición de corte** de la iteración, de modo que es importante considerar la consistencia entre la inicialización de las variables, la expresión y el bloque iterativo, para garantizar que la condición de corte se satisfaga en el momento adecuado. En particular es necesario evitar los ciclos infinitos.



Sub-programas

En la etapa de diseño de una solución, una estrategia fundamental es **dividir el problema en sub-problemas**.

Al escribir código en Pascal es importante mantener esta división **modularizando**.

Cada **sub-problema** puede ser implementado mediante un **sub-programa** de modo que el programa mantenga la **estructura modular** de la solución.

En Pascal los sub-problemas pueden implementarse mediante módulos llamados funciones o procedimientos.



Procedimientos

Un **procedimiento** es un subprograma que puede computar cero, uno o más valores y/o leer, procesar y mostrar datos.

Un procedimiento consta de un encabezamiento y un bloque ejecutable.

El encabezamiento incluye el **nombre del procedimiento**, la **lista de parámetros formales** y las **declaraciones locales**.

Las declaraciones pueden incluir a otros procedimientos o funciones, de modo que la definición de la estructura de un subprograma es recursiva.



Procedimientos

El nombre permite que el procedimiento se **invoque** desde el mismo procedimiento, desde otro sub-programa o desde el programa principal (considerando las reglas de alcance estático del lenguaje).

La **invocación** se realiza mediante una instrucción que incluye el nombre del procedimiento y una **lista de parámetros** a los que llamamos **reales** o **efectivos**.

Pascal permite que un procedimiento se invoque a sí mismo mediante una **llamada recursiva**. En este caso, es necesario que el programador establezca una condición de corte para que las llamadas no se produzcan indefinidamente.



Función

Una función es un **sub-programa** que computa una operación simple y devuelve un valor.

Está constituida por un **encabezamiento**, una **sección de declaraciones** y un bloque ejecutable.

El encabezamiento está formado por el **nombre de la función**, la **lista de parámetros formales** y el **tipo del resultado**.

Las declaraciones pueden incluir a otros procedimientos o funciones, de modo que la definición de la estructura de un subprograma es recursiva.

Las funciones no deberían tener **efectos colaterales**.



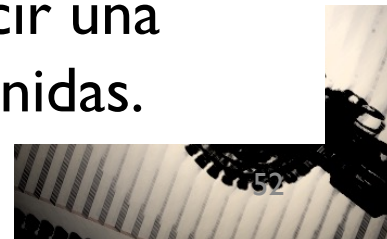
Función

Una función es invocada siempre desde adentro de una **expresión** en el programa principal, en otra función o procedimiento o desde sí misma (considerando las reglas de alcance estático del lenguaje).

La llamada se realiza mediante el nombre de la función seguida de la lista de parámetros reales, siempre en el contexto de una expresión.

El nombre de la función debería aparecer al menos una vez, en el código de la función, en la parte izquierda de una asignación, recibiendo el resultado que se va a retornar.

Una función puede invocarse a sí misma mediante una **llamada recursiva**. Nuevamente, el programador debe introducir una condición de corte para que las llamadas no sean indefinidas.



Parámetros

Los parámetros permiten que el sub-programa se comuniquen con el “exterior” (el exterior es el entorno del bloque ejecutable desde donde se invoca).

Cada parámetro que aparece en la lista del encabezamiento de la definición de la primitiva se llama **parámetro formal**.

La lista de parámetros formales tiene que coincidir en número y tipo de dato con la lista de parámetros reales (donde los reales son los que aparecen en la invocación).

Hay dos clases de parámetros: por **valor** o por **referencia**.



Parámetros por valor

Si el pasaje es por valor, **EN EJECUCION** se reserva una celda de memoria para el parámetro formal y en el momento de la llamada se inicializa con el valor del parámetro real.

Si durante la ejecución se modifica el valor del parámetro formal, el cambio no va a ser visible en el entorno de la llamada cuando el sub-programa termine.

Si un parámetro formal es por valor, el **parámetro real** asociado puede ser un valor literal, una variable, una llamada a función o una expresión.



Parámetros por referencia

Si el pasaje de parámetros es por referencia, el parámetro formal está precedido por la palabra reservada **var**.

Cuando se invoca el sub-programa, por cada parámetro por referencia se almacena EN EJECUCION la **dirección del parámetro real (LA FLECHA)**.

Cualquier referencia al parámetro formal es en realidad una referencia al parámetro real (que está en el entorno del bloque ejecutable donde fue invocado).

Si se modifica el parámetro formal, se modifica el parámetro real, de modo que al terminar el sub-programa, la modificación persiste.

El parámetro real asociado a un parámetro formal por referencia solamente puede ser el identificador correspondiente a una VARIABLE.

Parámetros por referencia

Si un parámetro formal es por referencia, el **parámetro real** asociado tiene que ser SIEMPRE **una variable**, NO puede ser una expresión ni un valor literal.

Aunque Pascal admite que las funciones tengan parámetros por referencia, no recomendamos hacerlo.



Declaraciones

Todo identificador tiene que estar **declarado** antes de ser usado.

El **bloque de declaraciones** de un programa o subprograma puede incluir:

- Etiquetas (no lo usamos)
- Constantes
- Tipos (no lo usamos)
- Variables
- Procedimientos y funciones



Ambiente de Referenciamiento

El **ambiente de referenciamiento** del **bloque ejecutable** del programa principal es el conjunto de identificadores que son **visibles** y pueden ser usados en ese bloque ejecutable.

Esta formado por identificadores:

- Predefinidos
- Globales

Cuando comienza la ejecución de un programa se reserva espacio en memoria para **las variables globales**, que van a existir durante **toda la ejecución**, aunque en algunos momentos pueden quedar **enmascaradas o tapadas (es decir, pueden estar vivas pero no ser visibles)** por declaraciones locales.

**SE DICE QUE UNA VARIABLE “ESTA VIVA”
CUANDO HAY UN LUGAR EN LA PILA PARA ELLA.**



Ambiente de Referenciamiento

El **ambiente de referenciamiento** de un **sub-programa** es el conjunto de identificadores que son **visibles** y pueden ser usados en el bloque ejecutable de ese sub-programa.

Está formado por identificadores:

- Predefinidos
- Globales
- No locales
- Locales

Cuando un sub-programa comienza a ejecutarse se reserva memoria para todos los parámetros formales y para las variables locales. Esta porción de memoria se llama **REGISTRO DE ACTIVACIÓN**.

Si es una función también se reserve un lugar para el resultado de la misma.

Cuando el sub-programa termina este espacio de memoria se libera.

Ambiente de Referenciamiento

Cuando se **referencia** un identificador en un bloque ejecutable, primero se busca entre las declaraciones locales, luego en las no locales, luego en las globales y por último entre los identificadores predefinidos.

Si no se encuentra en ninguno de estos niveles el compilador notifica un error.

Notemos que una declaración local puede **enmascarar** (tapar) a una declaración no local, global o predefinida.



Ambiente de Referenciamiento

Cualquier sub-programa podría acceder al ambiente global siempre que este esté visible.

SIN EMBARGO:

- La programación modular recomienda que un sub-programa solo acceda a sus parámetros formales y a las variables declaradas localmente.

Seguiremos esta recomendación, **solo usaremos las constantes y variables globales en el bloque ejecutable del programa principal.**

- LUEGO, cada sub-programa solo accederá a sus parámetros formales y a sus variables locales.

Recursividad

Hacer siempre el planteo recursivo

- Caso base o trivial: Se resuelve directamente, ya tenemos una solución o respuesta para ese caso sin acudir a la solución de una instancia menor. Debemos indicar cuándo estamos en el caso base y cuál es el resultado en este caso. Pueden ser varios casos triviales.
- Caso recursivo o general: Se resuelve a partir de tener la solución del mismo problema para una instancia menor (llamamos menor a la instancia que está mas cerca del caso base). Debemos indicar cuando estamos en el caso general, como obtenemos la solución de la **instancia actual** en función de la **solución de la instancia menor**, y debemos especificar claramente como se reduce la instancia.

EN EL PLANTEO, NO DEBE APARECER NINGUNA REFERENCIA A LA IMPLEMENTACION.

La Computadora

En el siglo X el matemático **persa Musa al-Juarismi**, le dio el nombre de **algoritmo** a una secuencia ordenada y finita de operaciones que permitían hallar la solución de un problema algebraico o numérico.

En 1645 **Blaise Pascal** inventa la **pascalina**, una de las primeras calculadoras mecánicas. Funcionaba a base de ruedas de diez dientes en las que cada uno de los dientes representaba un dígito del 0 al 9.

En 1801 el francés **Joseph Marie Jacquard**, utilizó un mecanismo de tarjetas perforadas para controlar el dibujo formado por los hilos de las telas confeccionadas por una máquina de tejer.



La Computadora



En 1822 **Charles Babbage** construye una máquina de propósito específico, calcular valores de polinomios y en 1837 diseña una **máquina analítica** de propósito general.

Aunque nunca llegó a construirla tuvo una influencia determinante en la creación de la computadora digital actual, porque introdujo el concepto de procesar programas almacenados en la memoria de un dispositivo electrónico para controlar los cálculos

La teoría que forma la base de la mayor parte del software moderno fue propuesta por **Alan Turing** en su ensayo de 1936, «Los números computables».

La Computadora



En 1946 en la **Universidad de Pensilvania** se construye la ENIAC (Electronic Numerical Integrator And Calculator), que fue la primera computadora electrónica de propósito general.

Esta máquina ocupaba todo un sótano de la Universidad, tenía más de 18.000 tubos de vacío, consumía 200 kW de energía eléctrica y requería todo un sistema de aire acondicionado.

Tenía la capacidad para realizar cinco mil operaciones aritméticas por segundo.



La Computadora

En 1951 comienza a operar la EDVAC en base a transistores, a diferencia de la ENIAC, el sistema de representación no era decimal, sino binario .

En 1954 se implementa el primer lenguaje de programación de alto nivel, Fortran.

En 1958 comienza la segunda generación de computadoras, los transistores se reemplazan por válvulas al vacío.

A partir de allí los inventos se suceden vertiginosamente.





Y... a ponerse a estudiar!
Que con suerte solamente... no alcanza!

Gracias por la hermosa compañía que fueron este 2024.
CUENTEN CONMIGO SIEMPRE. La profe Jessi

